NAGALAND UNIVERSITY

(A Central University Established by an Act of Parliament 1989)

SCHOOL OF ENGINEERING AND TECHNOLOGY

DEPARTMENT OF INFORMATION TECHNOLOGY

Kohima Campus, Nagaland – 797004



Assignment 5

ITB301: Object Oriented Programming

Submitted By:
Soushnabh Bagchi
Roll / ID: 2505264
Semester: 3rd Sem

Submitted To:
Er Sudipto Patowary
Assistant Professor
Dept. of IT

Date: November 5, 2026

1. Create an abstract base class Payment with a pure virtual function `processPayment(double amount)`. Implement derived classes CreditCardPayment, PayPalPayment and BitcoinPayment. Write a global function `void executeTransaction(Payment* paymentMethod, double amount)` that demonstrates runtime polymorphism.

Code:

```cpp
#include <iostream>

using namespace std;

class Payment {
    public:
        virtual void processPayment(double amount) = 0;
        virtual ~Payment() = default;
};

class CreditCardPayment: public Payment {
    public:
        void processPayment(double amount) {
            cout<<"paying amount: "<<amount<<" via Credit Card..."<<endl;
        }
};

class PayPalPayment: public Payment {
    public:
        void processPayment(double amount) {
            cout<<"paying amount: "<<amount<<" via PayPal..."<<endl;
        }
};

class BitcoinPayment: public Payment {
    public:
        void processPayment(double amount) {
            cout<<"paying amount: "<<amount<<" via Bitcoin..."<<endl;
        }
};

void executeTransaction(Payment* paymentMethod, double amount) {
    paymentMethod->processPayment(amount);
```

```
}

int main() {
    CreditCardPayment c;
    PayPalPayment p;
    BitcoinPayment b;

    executeTransaction(&c, 50);
    executeTransaction(&p, 100);
    executeTransaction(&b, 150);
    return 0;
}
```

Output:

```
paying amount: 50 via Credit Card...
paying amount: 100 via PayPal...
paying amount: 150 via Bitcoin...
```

2. Design a base class SmartDevice with virtual functions `turnOn()` and `turnOff()`, and a pure virtual function `getDeviceStatus()`. Derive SmartLight and SmartThermostat from it. Overload `turnOn()` in SmartLight to also dim to 50% by default and ensure your system can manage an array of different devices using `SmartDevice*` pointers.

Code:

```C++
#include <iostream>

using namespace std;

class SmartDevice {
    protected:
        bool device_status;
    public:
        virtual void turnOn() {
            cout<<"Turning Device On..."<<endl;
            device_status = true;
        }
        virtual void turnOff() {
            cout<<"Turning Device Off..."<<endl;
```

```
15              device_status = false;
16          }
17          virtual bool getDeviceStatus() = 0;
18          virtual ~SmartDevice() = default;
19  };
20
21  class SmartLight: public SmartDevice {
22      public:
23          int dim_percent;
24          void turnOn() {
25              cout<<"Turning SmartLight On (dimmed to 50%)..."<<endl;
26              device_status = true;
27              dim_percent = 50;
28          }
29          void turnOn(int dim_percent) {
30              cout<<"Turning SmartLight On (dimmed to
                "<<dim_percent<<"%)..."<<endl;
31              device_status = true;
32              dim_percent = dim_percent;
33          }
34          virtual bool getDeviceStatus() {
35              return device_status;
36          }
37  };
38
39  class SmartThermostat: public SmartDevice {
40      public:
41          virtual bool getDeviceStatus() {
42              return device_status;
43          }
44  };
45
46  int main() {
47      SmartLight light;
48      SmartThermostat thermostat;
49
50      // Array of base class pointers managing different devices
51      SmartDevice* devices[2];
52      devices[0] = &light;
53      devices[1] = &thermostat;
54
55      cout << "Turning all devices ON" << endl;
```

```
56      for (int i = 0; i < 2; i++) {
57          devices[i]->turnOn();
58          cout << "Status: " << (devices[i]->getDeviceStatus() ?
            "Active" : "Inactive") << endl;
59          cout << endl;
60      }
61
62      light.turnOn(80);
63
64      cout << "\nTurning all devices OFF" << endl;
65      for (int i = 0; i < 2; i++) {
66          devices[i]->turnOff();
67      }
68
69      return 0;
70  }
```

Output:

```
1   Turning all devices ON
2   Turning SmartLight On (dimmed to 50%)...
3   Status: Active
4
5   Turning Device On...
6   Status: Active
7
8   Turning SmartLight On (dimmed to 80%)...
9
10  Turning all devices OFF
11  Turning Device Off...
12  Turning Device Off...
```

3. Create a base class Report and derived classes FullTimeReport and ContractReport. Create a base class Employee with a virtual function `Report* generateReport()`. Derive FullTimeEmployee and ContractEmployee and use covariant return types so their respective `generateReport()` methods return the specific derived report types instead of the base pointer.

Code:

```
1   #include <iostream>                                                C++
```

```
2
3   using namespace std;
4
5   // --- Report Hierarchy ---
6   class Report {
7       public:
8           virtual void display() {
9               cout << "Generic Employee Report" << endl;
10          }
11          virtual ~Report() = default;
12  };
13
14  class FullTimeReport : public Report {
15      public:
16          void display() {
17              cout << "Full-Time Employee Report: Salary + Benefits
                    included." << endl;
18          }
19  };
20
21  class ContractReport : public Report {
22      public:
23          void display() {
24              cout << "Contract Employee Report: Hourly Rate + Invoice
                    included." << endl;
25          }
26  };
27
28
29  class Employee {
30      public:
31          virtual Report* generateReport() = 0;
32          virtual ~Employee() = default;
33  };
34
35  class FullTimeEmployee : public Employee {
36      public:
37          FullTimeReport* generateReport() {
38              return new FullTimeReport();
39          }
40  };
41
42  class ContractEmployee : public Employee {
```

```
43      public:
44          ContractReport* generateReport() {
45              return new ContractReport();
46          }
47  };
48
49
50  int main() {
51      FullTimeEmployee fte;
52      ContractEmployee ce;
53      FullTimeReport* ftReport = fte.generateReport();
54      ContractReport* ctReport = ce.generateReport();
55
56      cout << "Direct calls" << endl;
57      ftReport->display();
58      ctReport->display();
59
60      cout << "\nPolymorphic call via base pointer" << endl;
61      Employee* emp = &fte;
62      Report* genericReport = emp->generateReport();
63      genericReport->display();
64
65      delete ftReport;
66      delete ctReport;
67      delete genericReport;
68
69      return 0;
70  }
```

Output:

```
1  Direct calls
2  Full-Time Employee Report: Salary + Benefits included.
3  Contract Employee Report: Hourly Rate + Invoice included.
4
5  Polymorphic call via base pointer
6  Full-Time Employee Report: Salary + Benefits included.
```

4. Write a base class DatabaseConnection with a virtual function `void logConnectionStatus()`. Call this function inside the DatabaseConnection constructor. Create a derived class SecureConnection that overrides `logConnectionStatus()` to output "Secure Logged". Instantiate a SecureConnection object and explain why the output shows the base version instead of the derived version.

Code:

```C++
#include <iostream>

using namespace std;

class DatabaseConnection {
    public:
        virtual void logConnectionStatus() {
            cout<<"log connection established"<<endl;
        }
        DatabaseConnection() {
            logConnectionStatus();
        }
};

class SecureConnection: public DatabaseConnection {
    public:
        void logConnectionStatus() {
            cout<<"Secure logged"<<endl;
        }
        SecureConnection(): DatabaseConnection() {}
};

int main() {
    SecureConnection sc;

    return 0;
}
```

Output:

```
log connection established
```

Explanation: During the construction of SecureConnection, the base class constructor `DatabaseConnection()` executes first. At this point, the derived part of the object has not yet been initialized. To prevent accessing uninitialized derived members, C++ resolves all virtual function calls to the base class's implementation while inside the base constructor.